When Does LLM Control Pay Off? A Volatility–Cadence Regime Map for Edge Inference Management

AI-generated under human Research Director prompting for the CNSM 2026 "Agentic AI Researcher" experimental track

David Luo
Department of Electrical and Computer Engineering
University of Toronto
Toronto, Ontario
david.luo@mail.utoronto.ca

Alberto Leon-Garcia
Department of Electrical and Computer Engineering
University of Toronto
Toronto, Ontario
alberto.leongarcia@utoronto.ca

Abstract—An LLM controller consumes time, money, and energy each time it makes a decision. We ask when better decisions repay that cost for an edge inference service. On an NVIDIA Jetson AGX Orin, we compare three LLM controllers with three rule families over a pre-registered 3×3 grid of workload volatility and control cadence (20 paired seeds per cell). The rules are calibrated per cadence rather than left at default settings, and they win 19 of 21 cells. One hosted LLM wins two high-volatility cells, but by choosing cheaper, more accurate configurations rather than by improving SLO performance. Only one win survives correction for 21 comparisons, and both reverse when an adversarial audit introduces a volatility-informed static baseline. A mid-episode intent change also favors parameter-switched rules, while removing cost from the intent causes 2.7× over-provisioning. Local serial decoding reproduces every decision; continuous batching changes wording but not decisions; hosted reproducibility is partial or unavailable. Finally, one on-device 3B deliberation costs 14.1 J, equivalent to serving 2–12 requests and adding 1.6–9.3% energy overhead at 60 s cadence. The results show that LLM control should be justified against strong static and dynamic baselines, with the controller's own cost included. The code and supporting artifacts are available in a public GitHub repository.

I. Introduction

LLMs are increasingly used to analyze system state and plan control actions. Recent examples include chain-of-thought autoscaling [1], agentic management of edge inference [2], and intent-driven orchestration [3]. These systems are attractive because one model can interpret several signals and respond to an objective written in natural language. However, an LLM decision is not free. It delays the control loop, consumes tokens or local energy, and may fail or change across identical requests.

Prior evaluations often emphasize decision quality without asking whether its improvement exceeds this overhead. Some also compare learned controllers with default-configured rules, although careful calibration alone can remove much of a reported learning advantage [4]. We therefore treat the cost–benefit premise as the experiment. For one real edge service, we ask: RQ1, under which workload volatility and control cadence does an LLM controller outperform calibrated rules after its own operating cost is charged; RQ2, how reproducible are its decisions under local and hosted serving configurations; and RQ3, how much energy does one on-device deliberation use relative to serving a request?

This study makes three contributions. First, it reports a paired, pre-registered volatility–cadence surface with all 21 cells, both registered metrics, confidence intervals, and controller costs. Second, it measures how intent completeness, serving configuration, and determinism controls affect an LLM controller in practice. Third, it reports idle-subtracted energy for an on-device LLM deliberation and documents the adversarial audits that weakened or reversed positive results. The main finding is simple: calibrated rules win almost everywhere, and the apparent exceptions do not demonstrate better dynamic control.

II. System and Experimental Design

Managed service. A Jetson AGX Orin in MODE_30W serves short review texts through llama.cpp. The controller chooses the number of concurrent serving slots, $r \in [1, 6]$, and one of three model variants: tiny-1B4, lite-1B, or full-3B. Their measured accuracies are 0.722, 0.694, and 0.917; their single-stream p50 latencies are 0.40, 0.41, and 1.17 s. Thus lite-1B is a useful trap for the controller: it is less accurate than tiny-1B4 and no faster. Capacity measurements also show that independent processes on the shared GPU do not increase throughput. Continuous-batching slots do, although sublinearly: from one to six slots, full-3B increases from 0.74 to 1.60 rps and lite-1B from 1.78 to 4.70 rps.

The main sweep uses a discrete-event model whose service-time distributions come from these hardware measurements. Each controller decision is still a real model invocation, and its measured wall-clock latency advances

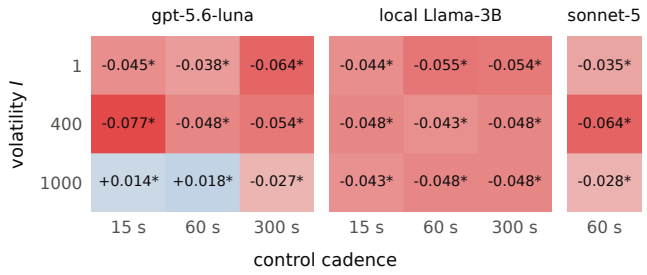


Fig. 1. Paired ΔU against the best calibrated rule in each cell (20 shared seeds). Positive values favor the LLM; $*$ marks a 95% CI that excludes zero. Sonnet ran only at 60 s under the registered budget rule.

the virtual clock. Each episode lasts 1800 s; the first 300 s are warm-up and excluded for every arm. Arrivals follow a two-state MMPP at mean 2.0 rps, fitted to a target index of dispersion I [5].

Score and comparisons. The registered utility is

$$U = -0.5V_{\text{off}} - 0.2A - 0.3C, \tag{1}$$

where V_{off} is the fraction of offered requests that finish late, are abandoned after 30 s, or remain at the episode boundary; A is the time-mean accuracy deficit; and C is mean slots divided by six. Higher utility is better, and a completed-only variant, U_{v2}, is computed in parallel. Both metrics, the hypotheses and falsification conditions, exclusion rules, budgets, and condition-cut order were registered before the runs.

The baselines are three structurally different rule families: static, queue-blind HPA-style, and queue-aware backlog-drain. Optuna calibrates every family separately at each cadence on held-out mixed-volatility traces using the same utility; deployed rules are not told I. The LLM arms are hosted gpt-5.6-luna, hosted claude-sonnet-5, and local Llama-3.2-3B. Luna and the local model run the full grid; sonnet runs the 60 s column under the registered budget rule. Every LLM receives the exact scored objective. Declared variants remove its cost clause, allow the model to choose its next wake time, or change the objective halfway through an episode. Workload seeds are shared across arms, and all reported margins are paired Student-t estimates ($n = 20$).

III. RQ1: Where Does LLM Control Pay?

Figure 1 reports the complete surface. Calibrated rules win 19 of 21 cells. The local 3B model never wins because it rarely behaves as a dynamic controller. In all 180 main-grid episodes, it selects an action at the first decision and never changes it. Across 9,180 decisions it emits only two actions; 83% select three slots of the dominated lite-1B variant. Sonnet also loses in every tested cell, including the cells where luna is positive.

Luna is better than the per-cell calibrated-rule envelope in two cells, both at nominal $I = 1000$: $+0.014$ $[+.008, +.020]$ at 15 s and $+0.018$ $[+.007, +.029]$ at 60 s. Both effects remain positive against each rule family and under either utility metric. Three qualifications narrow this result. First, decomposition shows worse SLO performance in both cells; the gains come from lower resource cost and higher model accuracy. Second, after Bonferroni correction over 21 comparisons, the 15 s result remains significant (corrected $p = 0.002$), but the 60 s result does not ($p = 0.064$).

Third, a stronger static baseline reverses both wins. An adversarial audit proposed tuning a static configuration on held-out $I = 1000$ traces. This volatility-informed baseline chooses one slot of full-3B and beats luna by -0.049 $[-.062, -.037]$ at 15 s and -0.062 $[-.076, -.049]$ at 60 s, where negative values mean luna is worse. The mechanism is that these traces are quiet about 94% of the time, but their bursts reach roughly 30 rps, beyond every available configuration. At nominal $I = 1000$, 8.8% [6.8, 10.8] of scored 15 s windows exceed maximum measured capacity; the corresponding fraction is 9.8% at $I = 400$. Extra capacity cannot recover these bursts, so minimal capacity with the most accurate model is the better policy. The LLM advantage is therefore relative to volatility-agnostic calibration, not evidence of better load management.

This result also contradicts the registered H1a prediction. The planned $I = 4000$ level was amended before any run because it was physically unreachable at this service scale. At the remaining moderate level, $I = 400$, luna's worst cell is 15 s: -0.077 $[-.088, -.066]$. At nominal $I = 1000$, realized volatility is heavy-tailed (pooled 30-window IDC 1921; bootstrap 90% band [687, 2987]), so we treat that label as a target rather than an exact realization.

The controller's own cost is operationally visible. At 15 s, luna averages about 119 calls, \$0.045, and 360 s of decision latency per 1800 s episode (means across the three volatility levels). This latency is charged to the plant. Luna has a 3.7% parse-failure rate, and 19.8% of raw main-grid attempts return empty completions and are retried;

TABLE I
Absolute utility at nominal $I = 1000$ ($n = 20$). The volatility-informed static baseline dominates every controller.

cadence	best rule (family)	gpt-5.6-luna	I-informed static
15 s	-0.558 (queue_aware)	-0.544	-0.495
60 s	-0.575 (static)	-0.558	-0.495
300 s	-0.575 (static)	-0.602	–

TABLE II
Controller cost per episode at 60 s (mean across volatility levels), with main failure modes. Local J/deliberation is measured in RQ3.

model	inv.	ktok	wall s	USD	fail%	empty%	J/delib.
gpt-5.6-luna	29	27.1	106	0.013	3.7	19.8	–
claude-sonnet-5	29	30.3	89	0.100	5.6	19.9	–
local Llama-3B	29	16.5	71	0.000	0.0	–	14.1

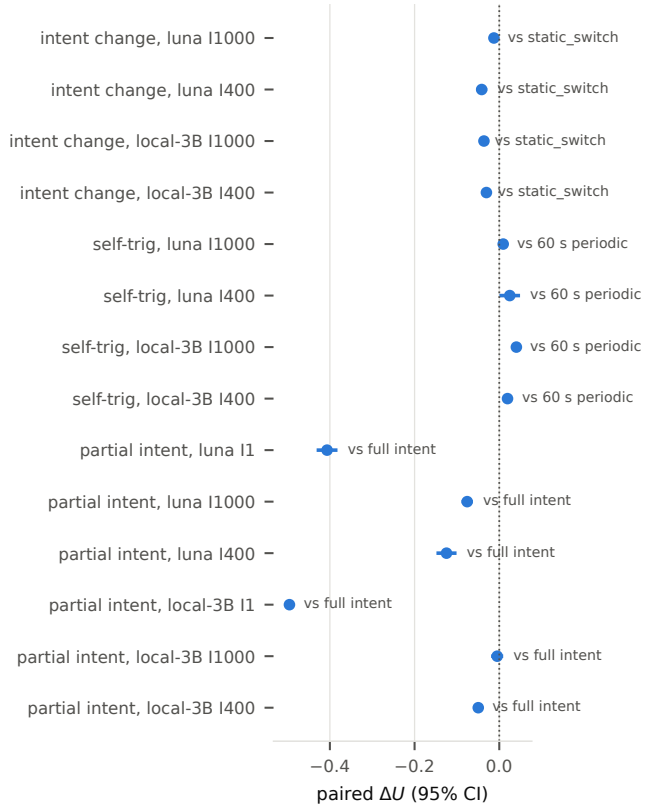


Fig. 2. Intent variants: paired ΔU and 95% CIs against the stated comparator.

sonnet's parse-failure rate is 5.6%, versus 0% locally. Total recorded hosted spend is $14.86.

IV. What the Intent Changes

Objective change. At $t = 900$ s, the SLO tightens from 3000 to 2000 ms and the utility weights change. LLMs receive a revised natural-language objective; rule controllers switch to parameters calibrated for the new objective. The LLM loses all four comparisons. For example, luna at $I = 400$ has $\Delta U = -0.042$ $[-.055, -.028]$. Natural-language adaptation provides no measured advantage over a mechanical parameter switch.

Self-triggered cadence. Allowing the model to choose its next wake beats its 60 s periodic twin in all four cells (+0.009 to +0.040) and uses 14–83% fewer calls. However, the SLO component is worse in every cell; 100–126% of the margin comes from reduced controller cost. The local model chooses one configuration and sleeps, while both hosted comparisons are sensitive to removing one seed. This is a cheaper approximation to static control, not better intervention timing. It still loses to ordinary static at $I = 400$ and to volatility-informed static at $I = 1000$.

Incomplete intent. Removing only the cost clause has the largest effect. At $I = 1$, luna increases from 2.20 to 6.01 mean slots and loses $\Delta U = -0.41$ $[-.43, -.38]$; the local model loses −0.50. Both models lose in every cell. The missing clause turns the controller into a peak-provisioner, and over-batching also worsens latency. Intent omissions are therefore behavioral changes, not benign defaults.

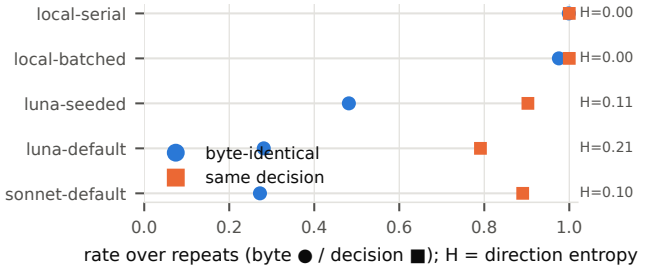


Fig. 3. Repeated byte-identical prompts over 30 real states and 30 repeats (20 for sonnet). H is normalized direction entropy.

TABLE III
Decision-identity rate by state stratum.

config	near-threshold	clear-overload	clear-calm
local-serial	1.000	1.000	1.000
local-batched	1.000	1.000	1.000
luna-seeded	0.867	0.948	0.926
luna-default	0.764	0.783	0.858
sonnet-default	0.853	0.923	0.928

V. RQ2: Decision Reproducibility

We replay 30 real states, stratified into clear-cut and near-threshold cases. The states come from main-grid logs replayed through the deterministic plant; 142 of 180 episodes reproduce exactly. We exclude 38 episodes containing API retries whose latency was not recorded, because byte-exact state recovery cannot be guaranteed.

The local serial configuration reproduces all 900 decisions. One response changes only its free-text rationale. Four-slot continuous batching also reproduces all 900 decisions, although byte identity falls to 0.976. Hosted responses are never byte-stable for every state. OpenAI's seed is useful but not deterministic: byte identity rises from 0.281 to 0.481, and decision identity from 0.791 to 0.903.

Claude-sonnet-5 exposes no usable determinism control in the tested interface. It rejects non-default temperature, top-k, and top-p with HTTP 400 and has no seed parameter (observed 2026-08-03; transcript committed). Thus the registered pinned configuration could not run. Table III shows that every hosted configuration is least stable near a decision boundary, where inconsistent choices matter most. One byte-identical state even produced both a scale-up and a scale-down decision across repeats. Hosted determinism is therefore best-effort and model-dependent; local decoding is the reproducible option in this experiment.

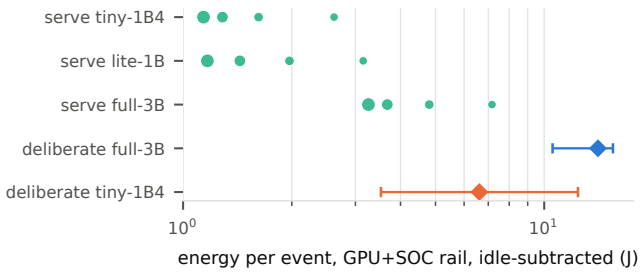


Fig. 4. Idle-subtracted GPU+SoC energy per event (log scale). Marker size denotes serving slots.

TABLE IV
What the main observations do and do not establish.

observation	supported reading	unsupported reading
19/21 rule wins	calibrated rules are stronger here	default rules always suffice
two luna wins	comparator knowledge changes the result	luna improves dynamic SLO control
reverse self-triggering wins	fewer calls reduce overhead	the model times interventions better
local decisions repeat	deterministic deployment is feasible	hosted decisions are reproducible
14.1 J per decision	control energy is measurable and material	deliberation always dominates serving

VI. RQ3: Deliberation Energy

The Jetson exposes GPU and SoC power on a fused rail, so per-process attribution is not possible; adjacent NVIDIA edge devices share this limit [6]. We instead characterize components separately using 10 Hz INA3221 samples and subtract idle power, following edge-LLM energy practice [7]. The 20 mA sensor resolution corresponds to about 0.4 W.

One full-3B deliberation with the main-grid prompt (531 input and 31 output tokens) costs 14.1 J. This batch estimate includes about 1.8 J of post-response ramp-down; the per-call mean without it is 12.3 J, and tiny-1B4 costs 6.6 J. Serving one request costs 1.14–7.17 J, so a full-3B decision equals roughly 2.0–12.4 served requests. The registered hypothesis of at least a tenfold difference is mostly falsified: only batched 1B-class service crosses that threshold.

At 60 s cadence and 2.0 rps, full-3B deliberation adds 1.6–9.3% to serving energy (tiny-1B4: 0.8–4.6%). Replacing a ramp-contaminated idle value found by the audit changes the ratio range to 1.9–12.1×, without changing which three configurations exceed tenfold. By comparison, conventional MAPE-K deliberation has been measured at 0.05% of system energy [8]. An LLM therefore raises deliberation energy by roughly two orders of magnitude, but it remains a minority of total energy here. Hosted energy is not observable from the client and is reported only as latency, tokens, and USD.

VII. Real-System Check

We replay three mechanically selected cells for 1800 s against the real llama.cpp service, including live controllers and drain-and-restart reconfiguration. Utility agrees within 2%: -0.516 emulated versus -0.519 real for the $I = 400$ rule, and -0.525 versus -0.535 for the local LLM. At the headline volatility, local-LLM utility is -0.639 versus -0.645. Offered counts match, and every real run is slightly worse, so the emulator does not inflate performance in these checks.

VIII. Discussion

The result is more specific than "rules beat LLMs." It shows how a plausible LLM-control advantage can disappear as the comparison becomes stronger. Luna first appears to win two cells against rules calibrated on a mixed-volatility workload. Utility decomposition then shows that these are not SLO wins. A workload-informed static comparator finally removes both aggregate wins. This sequence matters because each stage answers a different question: whether the LLM beats a deployable general-purpose rule, whether it manages latency better, and whether its behavior exploits structure that a simple policy could also exploit.

Three evaluation practices follow. First, static and dynamic baselines should be calibrated on the same objective as the LLM. A static baseline is not merely a weak lower bound: when bursts exceed all feasible capacity, refusing to react can be the correct policy. Second, aggregate utility should be decomposed. Here the apparent LLM gains come from model and capacity selection, while the SLO contribution is negative. Without decomposition, that distinction is hidden. Third, the controller must be included in the system boundary. At fast cadence, decision latency occupies one fifth of an episode; on-device energy equals several user requests; and hosted retries create additional delay.

The intent experiments also separate interface flexibility from control quality. Updating an English objective is operationally convenient, but it is not an advantage if a parameter switch produces better decisions. Conversely, the partial-intent result shows that a natural-language interface can make an omission harder to detect: the prompt remains fluent and valid while the controller silently changes the resource tradeoff. Intent validation should therefore check objective completeness before deployment, rather than rely on the model to infer missing costs.

Reproducibility requires a similarly precise unit of analysis. Byte identity is useful for forensic replay, but an operator may only require the selected action to repeat. Local batching separates these properties: it changes some rationales but preserves all actions. Hosted serving preserves neither property reliably, and its lowest action stability occurs near decision thresholds. A deployment claim should therefore state which identity is required and

measure it on difficult states, not only on clear cases.

Relation to prior work. ORACL evaluates chain-of-thought autoscaling against stock HPA settings [1]; our per-cadence calibration tests how the conclusion changes when rule baselines receive comparable tuning. The adaptive-control benchmark of [4] motivates this choice by showing that production-faithful configuration can remove reported learning advantages. E^3-Agent proposes an evolving agent for edge generative inference but reports no joules [2]; our measurement supplies the controller-side energy quantity needed for such a cost–benefit test. EcoMLS measures conventional MAPE-K overhead [8], providing the non-LLM reference point for RQ3. Finally, intent-driven orchestration [3] motivates treating intent completeness as an experimental variable rather than assuming that the prompt fully represents the operator's objective.

IX. Limitations and Research Process

The claims remain bounded to one service, device, task family, and prompt per arm. Hosted model IDs have no dated snapshots. The top volatility level is heavy-tailed. Energy is component-level in one power mode, and the main sweep does not model all controller–service contention on the shared GPU. The emulator is validated at three cells, not across the full surface. The volatility-informed static baseline also assumes advance knowledge of the workload class; it is a diagnostic upper envelope, not the originally registered deployable comparator.

Mechanical traceability checks arithmetic but cannot detect a biased comparison. Before publication, fresh-context agents were instructed to challenge every headline result. Those audits found the stronger static baseline, the frozen local controller, a 2.5% idle-power artifact, and two self-triggered specification gaps. Each issue was recorded rather than silently repaired. This is why the paper reports a reversal instead of an LLM win, and why we view adversarial design review as a necessary complement to reproducible artifacts. The code, experiment configurations, and result artifacts supporting this paper are available at https://github.com/dl423/cnsm2026-llm-control-regime-map.

Appendix

Disclosure Statement

This paper was generated entirely by AI systems, including the literature review, research ideation, experimental design and execution, data analysis, and manuscript preparation. The primary AI researcher was Claude Fable 5, accessed through Claude Code. Claude Opus 5 and Claude Sonnet 5 were also used to a lesser extent. OpenAI GPT-5.6 Sol, accessed through Codex, helped revise the manuscript for clarity and accessibility.

The research process began with a human prompt specifying the broad topic of "5G/6G Networks and Edge Computing Services.", asking the AI to survey current literature and identify gaps. Claude Code surveyed the literature and proposed several candidate research directions, from which the human Research Director selected one. It then conducted the experiments, analyzed the results, and drafted and revised the manuscript. Claude Code was run on a machine with access to Jetson Orin GPU and it has been informed about this. Afterwards, Codex was used for improving the readability of the draft. The human Research Director monitored progress and provided some intermediate direction through prompting the AI.

References

[1] H. Bai, M. T. Islam, M. Xu, and R. Buyya, "ORACL: Optimized reasoning for autoscaling via chain of thought with LLMs for microservices," arXiv:2602.05292, 2026.

[2] R. Bao, Y. Sun, K. Chen, B. Yang et al., "E^3-Agent: An executable and evolving agent for resource management of edge generative inference," arXiv:2605.27428, 2026.

[3] Y. Sun, B. Guo, Q. Huang, Y. Feng et al., "Generative intent prediction agentic AI empowered edge service function chain orchestration," arXiv:2601.13694, 2026.

[4] G. Zhang, X. Sun, Y. Zhao, W. Chu et al., "When does deep RL beat calibrated baselines? A benchmark study on adaptive resource control," arXiv:2605.26418, 2026.

[5] N. Mi, G. Casale, L. Cherkasova, and E. Smirni, "Injecting realistic burstiness to a traditional client-server benchmark," in Proc. 6th International Conference on Autonomic Computing (ICAC), 2009.

[6] D. Panigrahy and K. Tyagi, "The energy blind spot: NVIDIA's flagship edge AI hardware cannot support process-level energy attribution," arXiv:2605.27599, 2026.

[7] M. Arya and Y. Simmhan, "Understanding the performance and power of LLM inferencing on edge accelerators," arXiv:2506.09554, 2025.

[8] M. Tedla, S. Kulkarni, and K. Vaidhyanathan, "EcoMLS: A self-adaptation approach for architecting green ML-enabled systems," in IEEE 21st International Conference on Software Architecture Companion (ICSA-C), 2024, pp. 230–237.